

Received 6 October 2025, accepted 5 November 2025, date of publication 12 November 2025,
date of current version 21 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3632207

RESEARCH ARTICLE

Packet Inspection Transformer: A Self-Supervised Journey to Unseen Malware Detection With Few Samples

KYLE STEIN¹, (Student Member, IEEE), GUILLERMO FRANCIA III², (Senior Member, IEEE),
EMAN EL-SHEIKH², (Member, IEEE), AND ANDREW ARASH MAHYARI^{1,3}, (Member, IEEE)

¹Department of Intelligent Systems and Robotics, University of West Florida, Pensacola, FL 32514, USA

²Center for Cybersecurity, University of West Florida, Pensacola, FL 32514, USA

³Florida Institute for Human and Machine Cognition (IHMC), Pensacola, FL 32502, USA

Corresponding author: Kyle Stein (ks209@students.uwf.edu)

This work was supported in part by the University of West Florida (UWF) Argo Cyber Emerging Scholars (ACES) Program funded by the National Science Foundation (NSF) CyberCorps Scholarship for Service (SFS) Award under Grant 1946442.

ABSTRACT As networks continue to expand and become more interconnected, the need for novel malware detection methods becomes more pronounced. Traditional security measures are increasingly inadequate against the sophistication of modern cyber attacks. Deep Packet Inspection (DPI) has been pivotal in enhancing network security, offering an in-depth analysis of network traffic that surpasses conventional monitoring techniques. DPI not only examines the metadata of network packets, but also dives into the actual content being carried within the packet payloads, providing a comprehensive view of the data flowing through networks. While the integration of advanced deep learning techniques with DPI has introduced modern methodologies into malware detection and network traffic classification, state-of-the-art supervised learning approaches are limited by their reliance on large amounts of annotated data and their inability to generalize to novel, unseen malware threats. To address these limitations, this paper leverages the recent advancements in self-supervised learning (SSL) and few-shot learning (FSL). Our proposed self-supervised approach trains a transformer via SSL to learn the embeddings of packet content, including payload, from vast amounts of unlabeled data by masking portions of packets, leading to a learned representation that generalizes to various downstream tasks. Once the representation is extracted from the packets, they are used to train a malware detection algorithm. The representation obtained from the transformer is then used to adapt the malware detector to novel types of attacks using few-shot learning approaches. Our experimental results demonstrate that our method achieves classification accuracies of up to 94.76% on the UNSW-NB15 dataset and 83.25% on the CIC-IoT23 dataset.

INDEX TERMS Deep packet inspection, few-shot learning, malware classification, malware detection, self-supervised learning.

I. INTRODUCTION

With the continuing expansion of the digital landscape, the significance of malware detection becomes increasingly important. With the modernization of sophisticated cyber attacks, traditional security measures often fall short in providing adequate protection. Traditional firewalls and

antivirus software often prove inadequate since they primarily rely on predefined packet header rules through conventional packet filtering and known malware signatures, leaving them vulnerable to new and sophisticated cyber attacks that do not match existing patterns or behaviors [1], [2]. Deep Packet Inspection (DPI) is a well-known tool for the analysis of network packets in the security of networks [3], [4]. DPI goes beyond conventional network monitoring tools by examining not only the five-tuple of network packets, such as source

The associate editor coordinating the review of this manuscript and approving it for publication was Tiago Cruz¹.

and destination Internet Protocol (IP) addresses, but also inspects the data payload within each packet. This in-depth analysis allows for a more comprehensive understanding of the nature of the traffic passing through a network, enabling the identification of malicious activities that might otherwise go unnoticed.

With recent advances in deep learning and artificial intelligence [5], [6], [7], the integration of these techniques into network traffic analysis and DPI has become more prevalent. The dynamic nature of cyber attacks, characterized by constantly evolving malware, necessitates a shift from traditional, signature-based detection methods to more adaptive and intelligent systems. It is estimated that 560,000 new pieces of malware are detected daily and more than 1 billion malware programs exist [8]. Deep learning, with its capability to model complex patterns and adapt to evolving information through neural networks, presents a promising approach to addressing the challenges of modern malware detection. With this, cybersecurity systems can analyze data in real time, identifying threats based on behavioral patterns rather than relying solely on known signatures.

DPI faces several *critical challenges* that limit its efficiency and effectiveness. *First*, most existing methods focus primarily on statistical-based features or inspecting the few first bytes of raw packet payload data, restricting their ability to capture deeper insights. *Second*, supervised learning approaches struggle to generalize beyond known attack types, as they learn representations tailored to the classes labeled in the training data [13], [21], [22], resulting in poor performance when faced with novel threats or emerging attack patterns. *Third*, these methods heavily rely on large amounts of annotated data, excelling in detecting malware types with abundant labeled samples but performing poorly on malware with limited or no annotations.

By leveraging few-shot learning techniques, we enable these models to adapt seamlessly to novel, unseen malware threats, setting a new standard in malware detection. The key contributions of this paper are as follows:

- **Whole Payload Inspection:** Transformers have already revolutionized natural language processing and computer vision [5], thanks to their deep, interconnected architectures that excel at capturing long-range dependencies in sequential data. In this paper, we harness the unparalleled capabilities of transformers to redefine malware detection, moving beyond traditional methods that rely on statistical-based features and IP tracking, which are prone to false alarms and missed threats. Moreover, transformers possess the extraordinary ability to analyze entire payloads in their full context unlike convolutional neural network (CNN)-based approaches, which are limited to inspecting only the first 500 bytes of payload data.
- **Generalization to Unseen Malware Types:** Self-Supervised Learning (SSL) is a paradigm that empowers models to extract deep, meaningful representations from vast amounts of unlabeled data—eliminating the need

for costly and labor-intensive manual annotation [5], [23], [24]. By combining the transformers' context-aware learning with the power of SSL, we train models on massive unannotated payload datasets, equipping them to decode intricate patterns within both benign and malicious payloads. This enables the model to identify complex patterns across large-scale benign and malicious payloads and achieve generalization to unseen malware types.

- **Learning Unseen Malware Patterns from a few Annotated Payloads:** Traditional supervised learning-based payload inspection methods rely on vast amounts of annotated data for each malware type [13], [14], [21]. The challenge becomes even more severe when novel malware variants appear, as limited samples make it nearly impossible to train conventional models effectively. While transformers, pre-trained on unannotated data, offer promising generalization capabilities, they inherently lack the ability to recognize unseen malware types. To bridge this gap, we integrate few-shot learning techniques, enabling the model to rapidly adapt to both previously encountered and entirely novel malware strains using minimal labeled examples. This approach dramatically enhances detection efficiency and accuracy, allowing AI-driven cybersecurity systems to identify and neutralize emerging threats—a crucial advancement in the fight against ever-evolving cyber attacks.

The structure of this paper is as follows: Section II reviews related work, while Section III provides background on network packets, self-supervised learning, and few-shot learning. Section IV describes the datasets and pre-processing steps. Section V details the model detection assumptions. Section VI details the proposed model architecture and training protocols. Section VIII presents the evaluation and ablation results, and Section X discusses experimental results using AES and Fernet encryption. Section XI provides further discussion, and Section XII concludes the paper.

II. RELATED WORK

In this section, we discuss related works employing modern techniques for detecting and mitigating cyber threats, focusing on approaches that leverage machine learning, deep learning, attention mechanisms, and FSL to improve accuracy in various detection tasks, summarized in Table 1.

A. MACHINE/DEEP LEARNING ATTACK DETECTION

Aceto et al. [4] delve into the application of deep learning in classifying mobile network traffic, highlighting the capability of deep learning systems to process network traffic independently of port information, thus distinguishing between different app-generated traffic effectively. In [9], researchers explored various methodologies, including self-taught learning and softmax regression, utilizing the NSL-KDD dataset for benchmarking. The research applies supervised learning to analyze over 41 statistical attributes, excluding payload byte details. Doshi et al. [10] focus on utilizing machine

TABLE 1. Comparative summary of related works and the proposed approach.

Reference	Data	Features	Method/Architecture	Unseen Attack?	Key Observations
Aceto et al. [4]	Mobile Network Traffic	Statistical Attributes	Deep Learning (various)	✗	Demonstrates deep learning for mobile traffic classification; does not address unseen malware.
Patheja et al. [10]	NSL-KDD	Statistical Attributes	Self-taught Learning, Softmax Regression	✗	Uses classic supervised learning with limited features; raw byte payloads not considered.
Doshi et al. [11]	Custom IoT DDoS dataset	Statistical Attributes	ML Ensemble + Simple NN	✗	Focuses on DDoS detection only; lacks generalization to unseen classes.
Almaraz-Rivera et al. [12]	IoT Traffic	Packet Counts, Bytes, Durations	ML + DL (DDoS)	✗	Primarily scale-based features; excludes raw payload bytes and novel threats.
Stein et al. [13]	CAN Bus (Automotive)	CAN Payload (8 bytes)	RNN-based Supervised	✗	Demonstrates viability in automotive domain; payload shorter than typical IP packets.
Saiyed et al. [14]	Custom IoT DDoS dataset	Traffic-Level Statistics	Ensemble (CNN + LSTM)	✗	Excels at DDoS detection but does not target new/unseen threats.
He et al. [15]	Malware PE/APK	RGB-Transformed Binaries	ResNeXt w/ Attention	✗	Employs attention for malware classification; requires labeled data for known threats.
Zhou et al. [16]	Website Fingerprinting	Packet Traces (TLS)	Transformers (Self-Attention)	✗	Improves WF classification via attention; does not address limited labeling or novel classes.
Ravi & Alazab [17]	Network Traffic	Extracted Features + CNN	Attention-based CNN	✗	Enhances classification accuracy but lacks handling of unseen malware classes.
Yu et al. [18]	Custom Flow Data (imbalanced)	Flow-Level Statistics	CNN + FSL	✗	Alleviates class imbalance with FSL; only uses flow features, no raw payload data.
Lu et al. [19]	IoT Intrusions	Flow-Level Statistics	CNN + FSL	✗	Addresses zero-day attacks; relies on high-level features, ignoring payload details.
Zhou et al. [20]	Industrial Cyber-Physical	CNN-Based Feature Vectors	Siamese CNN + FSL	✗	Works well for anomaly detection but excludes longer packet contexts or raw bytes.
Stein et al. [21]	Network and IoT Intrusions	Raw Packet Bytes	Supervised Transformer + FSL	✓	Generalizes well in FSL tasks but assumes there is enough labeled data for supervised pre-training.
Proposed Method	Network and IoT Intrusions	Raw Packet Bytes	Self-Supervised Transformer + FSL	✓	Learns embeddings from raw unlabeled packet data; adapts quickly to novel/unseen attacks with few samples.

learning to detect Distributed Denial of Service (DDoS) attacks, using a combination of four machine learning algorithms and a simple neural network to differentiate between normal and DDoS traffic in Internet of Things (IoT) environments. They emphasize the creation of a new dataset for algorithm testing, noting the significant size and imbalance between malicious and benign data samples.

In another study [25], machine learning is employed for IP traffic classification within a 4G network context, with the researchers generating a dataset specific to 4G traffic and applying machine learning algorithms to its packet content. This research specifically excludes analysis of malicious traffic, focusing instead on general IP traffic flow. Similarly, Almaraz-Rivera et al. [11] investigated the use of machine and deep learning methods for detecting DDoS attacks, relying on statistical data like packet counts, bytes, and the duration of transactions rather than raw packet bytes.

Stein et al. [12] demonstrated that CAN payloads in vehicles can be classified as benign or malicious using a supervised deep learning approach with a Recurrent Neural Network (RNN). However, it's important to note structural differences: CAN bus messages have an 8-byte payload limit, while IP network packets typically support up to 1500 bytes. Recent works include the DEEPSHIELD system, an ensemble learning approach designed to detect solely DDoS attacks in IoT environments by integrating Convolutional Neural

Networks (CNNs) and Long Short-Term Memory (LSTM) networks with a network traffic analysis module [13]. While these methods show the ability to generalize known attack types, it remains a critical consideration to detect novel and unseen types entering a network. Furthermore, these methods predominantly rely on statistical or high-level features of network traffic, overlooking the potential of raw packet-level byte data, which can provide more intricate representations of the network content.

B. ATTENTION-BASED ATTACK DETECTION

Researchers have recently proposed several innovative approaches for malware detection and classification. For example, ResNeXt+ [14] integrates various plug-and-play attention mechanisms into the ResNeXt architecture to improve the identification of malware by converting malware PE/APK files into RGB information. The authors highlighted the model's effectiveness across multiple datasets and various scenarios. In another notable work, Zhou et al. [15] proposed the use of Transformers in the cybersecurity domain, showcasing advancements in website fingerprinting over previously used CNNs by effectively capturing long-term dependencies in traffic traces through self-attention mechanisms. This approach has demonstrated promising results in both closed-world and open-world scenarios, providing enhanced performance in WF tasks. Similarly,

Ravi and Alazab [16] proposed an attention-based CNN approach for robust malware classification. The method enhances malware detection by integrating attention mechanisms into the CNN architecture, helping improve feature extraction and classification accuracy. Researchers in [20] employ a Large-Language Model (LLM) as a feature extracting backbone to extract low-dimensional embeddings of raw packet data for downstream classification tasks. Zhang et al. [26] addressed the challenge of detecting and classifying ransomware by proposing a static analysis framework based on a partitioning strategy combined with a self-attention-based CNN (SA-CNN) and a bi-directional self-attention network. While these attention-based approaches tend to improve performance over classic deep-learning techniques, a common limitation remains: the reliance on large labeled datasets. The problematic nature of accessing extensive labeled data, as well as the models' limited ability to adapt and classify previously unseen malware classes, continues to be an overlooked challenge in the field.

C. FEW-SHOT LEARNING ATTACK DETECTION

FSL's aim is to learn from limited amounts of labeled data. This is particularly important for intrusion detection, where obtaining large, labeled datasets for new or emerging attack types is difficult. The cyber domain has begun to implement FSL techniques to reduce the need for labeled datasets. For example, a FSL method was proposed to address the data imbalance issue in network intrusion detection by extracting statistical-based network flow features using a CNN and performing FSL analysis downstream [17]. Similarly, researchers implemented a Siamese CNN to extract features for anomaly detection in industrial cyber-physical systems [19]. The CNN features were once again used for FSL downstream to measure similarities between samples, enabling effective detection of anomalies with minimal labeled data. Lastly, another proposed FSL-based intrusion detection model tailored for IoT environments addressed the challenge of limited labeled data and emerging, zero-day attacks [18].

Although these works produce promising results, they primarily rely on simple CNN-based feature extractors. Simple CNN-based extractors often struggle to capture long-term dependencies and subtle variations within packets, limiting their ability to generalize to unseen attack types effectively. SSL addresses these shortcomings by learning robust and generalizable feature representations based off attention. Additionally, prior FSL methods simulate unseen attacks by re-splitting known classes into different training and testing subsets. The model still has exposure to all of the classes distributions, just with limited examples from each. In contrast, our approach completely excludes one of the attack classes during SSL pre-training and then evaluates the model's ability to detect the unseen class along with the seen classes in the FSL evaluation step. This approach is similar to our previously published work [20], where one

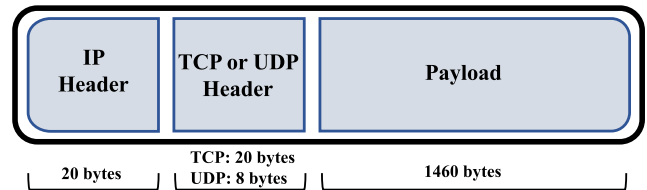


FIGURE 1. Network packet structure.

class is explicitly excluded from training to simulate the unseen scenarios. However, in our previous work we relied on **supervised** learning to pre-train the LLM on scratch from known malware types. Our work in this manuscript differs since we employ SSL techniques to pre-train the transformer on **unlabeled** data. This helps demonstrate that SSL can learn meaningful representations from the packet data without relying labeled data, a crucial step in real world applications. Overall, this approach ensures that the untrained class remains completely unseen during the training phase.

III. BACKGROUND

The proposed methodology implements multiple learning criterion throughout the study. In this section, we discuss the necessary preliminary information that sets the foundation for understanding our approach.

A. FUNDAMENTALS OF NETWORK PACKETS

Network packets play a critical role in the transmission of data across networks. These packets contain information for routing and delivery, including the source and destination IP addresses. IP addresses identify the sending and receiving devices, and the source and destination ports, which specify the sending and receiving applications. These elements form what is known as the five-tuple: Source IP Address, Destination IP Address, Source Port, Destination Port, and Transport Protocol. The Transport Protocol component of the five-tuple specifies the type of protocol used for the connection. Transmission Control Protocol (TCP) and User Datagram Protocol (UDP), being the two most common transport layer protocols [21], are the focus in this study. TCP is known for its reliability and ordered delivery of packets, establishing a connection-oriented session between communicating devices. TCP guarantees that data is received exactly as it is sent, making it ideal for applications where accuracy is necessary, such as web browsing and email transmission [27], [28]. UDP, on the other hand, is a connectionless protocol known for its low-latency and efficient data transfer, without the same reliability guarantees as TCP. This is commonly used in streaming media, online gaming, and real-time communications where speed is more crucial than perfect delivery [27], [28]. These two protocols account for the majority of network traffic at the transport layer.

Given that the five-tuple function is to identify connections, it is important to note that packets also encompass

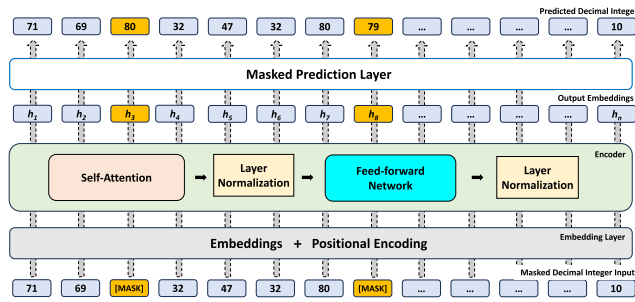


FIGURE 2. Self-supervised learning with masked language modeling.

network payloads. These payloads carry the substantive content being transmitted, including text, images, and files, from the source to the destination. Malicious actors often embed malware within these payloads, exploiting the data transfer process to compromise targeted systems [11], [21], [22], [29]. In a given network packet, the maximum payload size is up to 1460 bytes. Payloads that exceed this limit are fragmented to ensure the complete transmission of data across the network, preventing any portion of the content from being lost during transit. Notably, the fragmentation of network payloads depends on the capacity of the network segment, and these fragments can be reassembled either at the endpoint or by intermediate devices. Furthermore, transformer-based architectures are well-suited for processing fragmented payloads since the self-attention mechanism allows the model to capture dependencies across separate fragments, therefore integrating information that is dispersed over multiple packets. Initially, raw packet and payload data exists in binary form and is then converted to hexadecimal format for various purposes. The hexadecimal representation offers a human-readable form, simplifying the process of data inspection and analysis for network experts. Fig. 1 shows a basic diagram of the header and payload bytes with a Maximum Transmission Unit (MTU) of 1500 bytes. The MTU represents the largest size of an Ethernet packet or frame that can be transmitted over a network. *Furthermore, it is important to note that DPI operates under the principle that the majority of data transmissions occur without encryption, which enables direct inspection of packet content* [30], [31].

B. SELF-SUPERVISED LEARNING

SSL learns a representation of the data via a pretext task. The pretext task is defined depending on the application and the type of the dataset. For example, the pretext task in natural language processing (NLP), which involves learning patterns from a sequential dataset (i.e. sentences), is predicting masked words in a sentence. Through prediction of the masked words, the internal transformer architecture learns to extract distributed representations of entire sentences without the masked word, such that the representation carries enough information to predict what the missing word is. Operating on the principle that the structure of the data

can provide insights in an unsupervised manner, it seeks to reduce dependence on labeled datasets and learn to recover whole or parts of the features of its original input [23], [24]. Masked language modeling (MLM) is a key component in SSL and introduced a novel methodology to utilize unlabeled data [5]. MLM involves predicting a masked input within a sequence, allowing the model to consider context from both the left and right sides of any masked input. The bidirectional attention grants the model complete access to all surrounding tokens, enhancing the ability to understand the full context of the sequence [5]. We extend this concept to understand the relationship between the bytes in a packet sequence based on the surrounding context. This approach enhances the model's ability to generalize from the unlabeled data to unseen examples, improving its predictive accuracy on the raw packet bytes

As shown in Fig. 2, the MLM process begins with masked input values, where specific positions (i.e. [MASK]) are hidden from the model. The input values are passed through an embedding layer combined with positional encoding to preserve the order sequence. The transformer encoder then processes the masked sequence through self-attention and feed-forward networks, followed by layer normalization. The resulting output embeddings are sent to a masked prediction layer, which predicts the missing values based on the learned contextual representations. The predicted values are compared with the actual masked values using a cost function, and the resulting loss is backpropagated through the model to update all parameters. This modeling approach is effective for tasks demanding a global understanding of an entire sequence's context.

C. FEW-SHOT LEARNING

Similar to how humans are able to recognize objects after observing them once, FSL is a specialized approach designed to enable models to learn and generalize from very limited amounts of labeled data [32], [33], [34]. In FSL, a classifier needs to accommodate for new classes not seen in training, given only few examples from each of those classes. One-shot learning is a form of few-shot learning where the model needs to learn given only one example per class [32], [35]. These algorithms must be able to extract rich contextual understanding from the limited amount of seen examples to generalize to new and unseen instances.

Snell et al. [33] introduced Prototypical Networks for FSL. The main idea behind this research was to show that data points belonging to the same class should be close in distance to each other in an embedding space. The authors suggested that each class should have a single representative or prototype, and this prototype is the mean of all data points in that class once they have been projected into a shared embedding space. Support and query sets are then used to simulate the few-shot learning scenario during training and testing. The support set consists of a small number of labeled examples for each class and are used to define the

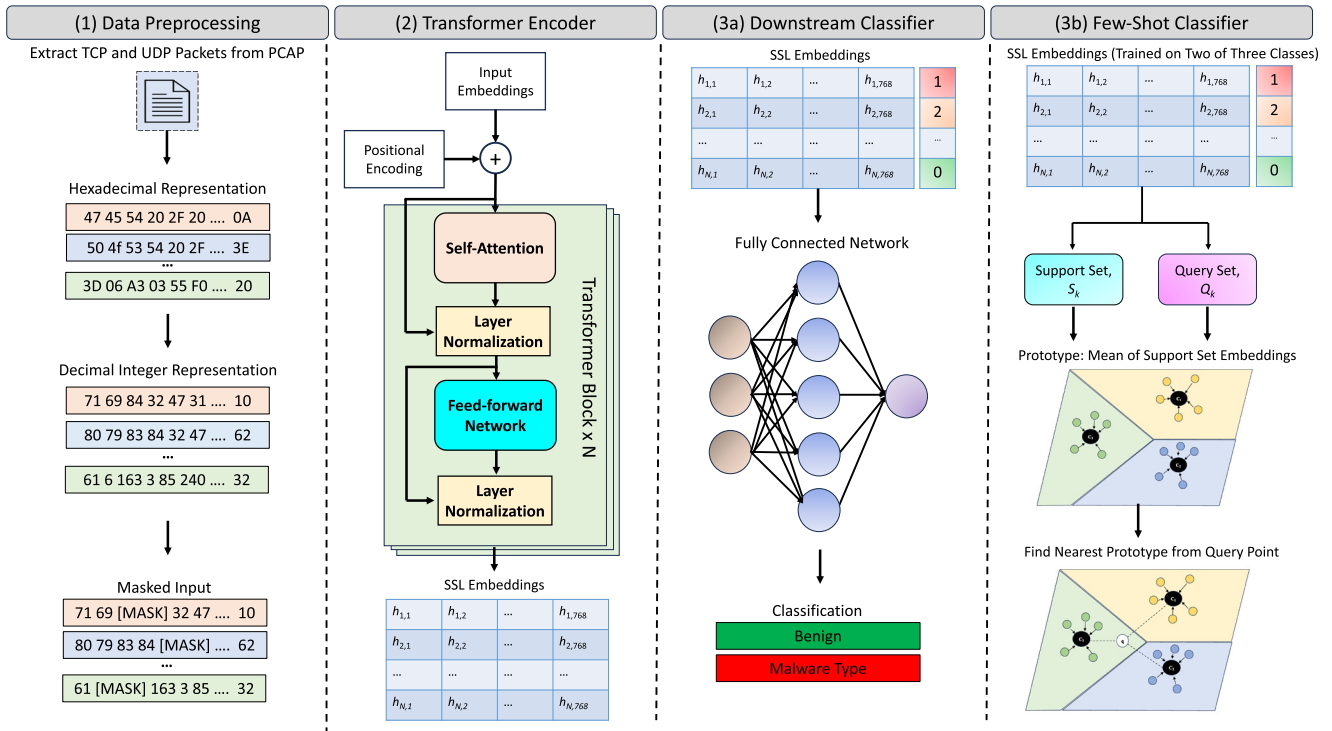


FIGURE 3. The overall architecture of the proposed packet detection algorithm.

prototype by computing the mean vector of the embedded points for each class. The query set contains unlabeled examples that the model needs to classify. These examples are different from those in the support set and are used to evaluate the performance of the model. Once the query and support sets are embedded into the space, the model computes the Euclidean distance between each query point and the prototypes. The query points are then classified based on the nearest prototype in the space.

IV. DATA

In this section, we discuss the characteristics and pre-processing of two distinguished and well-recognized datasets: UNSW-NB15 [36] and CIC-IoT23 [37]. These two datasets were chosen due to their comprehensive nature and relevance in the field of cybersecurity, particularly in the context of network intrusion detection and IoT security. Both datasets are available from their official websites, and the pre-processing steps outlined below allow for full reproducibility.

A. UNSW-NB15

The UNSW-NB15 dataset was developed to address the need for more extensive public intrusion detection network datasets. Although benchmark datasets such as the KDD-CUP99 [38] were prominently used for many years, the UNSW-NB15 tackles the shortcomings these older datasets have, such as redundancy and missing records. The UNSW-NB15 comprises 100 GB of raw network traffic captured in

PCAP format, featuring a balanced mix of real and simulated modern attack scenarios. For practicality and to ensure efficiency throughout the study, from the available 100 GB of PCAP data, a subset of 4 GB was selected. This subset includes diverse traffic scenarios, representing a comprehensive coverage of network attacks and behaviors that are available in the data.

The primary input to the model consists of packet bytes, represented as decimal integers. To ensure a balanced dataset and minimize bias, an equal number of malicious packets from three selected attack types are randomly sampled. This step is crucial to avoid overrepresentation of any single attack, which could otherwise lead to an increase in false positives or negatives during classification. Duplicate packets are also removed to prevent unintended biases. For multi-class experiments, we focus on three attack categories—Fuzzers, Exploits, and Generic—due to their payload complexity, distinguishing them from simpler, volume-driven attacks like Distributed Denial of Service (DDoS).

B. CIC-IoT23

The CIC-IoT23 dataset is a comprehensive resource designed to explore vulnerabilities in Internet of Things (IoT) security. It includes seven broad categories of attacks—DDoS, Brute Force, Spoofing, Denial of Service (DoS), Recon, Web-based attacks, and Mirai malware—further divided into over 30 specific IoT-focused attack types. The dataset represents detailed network activity captured from a diverse IoT environment with 105 unique devices, making it a rich

source of PCAP files that log both benign and malicious activities.

The pre-processing for CIC-IoT23 closely parallels that of UNSW-NB15. Transport layer payloads are extracted from TCP and UDP packets in randomly selected PCAP files. These extracted bytes are initially converted to hexadecimal format, with duplicates removed to maintain data integrity. The cleaned data is then converted into decimal integers, and each malicious class is assigned a unique label. To maintain balance in the dataset and prevent model bias, we randomly sample an equal number of malicious entries from each attack class. For this study, we narrow our focus to three specific attack types from CIC-IoT23: Backdoor Malware, Vulnerability Attacks, and Brute Force Attacks. These attacks rely on the content of the packet to execute their malicious objectives, making them comparable to the chosen attacks from the UNSW-NB15 dataset.

V. THREAT DETECTION ASSUMPTIONS

Our model focuses on active, traffic-based detection by analyzing the content of network packet bytes. The primary goal is to identify malicious content before execution by inspecting packet byte sequences during transmission. To clarify the scope and limitations of this work, we explicitly define the assumptions associated with our approach.

We assume access to active network traffic, specifically focusing on packet content from TCP and UDP transmission protocols. A significant portion of data transmissions occurs in plaintext [30], [31], which permits direct inspection of packet content. We also assume that analyzing the complete payload, even when it is fragmented across multiple packets, is sufficient for detection. With these assumptions, the proposed method offers several capabilities. By leveraging SSL during training, the model can learn robust representations of network bytes without labels, enabling the detection of malware content early in active traffic. Furthermore, the proposed method generalizes to unseen malware classes using FSL, where limited labeled examples are sufficient to classify introduced attack types. The reliance on packet data also allows for efficient, real-time processing, making the approach scalable to high-throughput network traffic. Lastly, since the model assumes plaintext packet bytes, its effectiveness may diminish for encrypted traffic. To mitigate these risks, we evaluate the impact of encryption on detection performance by applying two symmetric encryption methods and repeating the method.

VI. SYSTEM ARCHITECTURE AND MODEL TRAINING

In this section, we describe the system architecture, training, and evaluation of our model for network traffic classification. Our method operates in multiple stages: we first pre-train a byte-level transformer encoder with SSL through a MLM objective *from scratch* on unlabeled packet bytes, producing packet embeddings from the final hidden states of the transformer. Next, to validate the learned embeddings, we implement a lightweight downstream classifier on top of the frozen

encoder for binary and multiclass malware detection and classification under the assumption that sufficient labeled data exists for conventional downstream classification. The overall architecture of the proposed method can be referred to in Fig. 3.

A. TRANSFORMER ENCODER

Transformer-based models have been powerful in natural language processing tasks due to their ability to uncover the complex relationships between words in sequences of textual data. Although individual bytes in a network packet may not inherently carry semantic meanings like words in a sentence, their sequences, patterns, and relative positions can encapsulate information about the nature of the packet. Leveraging transformer's capability to capture contextual patterns, we show that it can be effectively applied to discern distinctions in packet sequences. This process is powered by the self-attention mechanism [7], enabling each byte in the input sequence to reference and weigh other bytes within that sequence while formulating its output representation. The mechanism determines a significance score for each byte in the sequence for the degree of attention it should receive. The transformer encoder architecture can be divided into these main parts:

1) EMBEDDING LAYER

The embedding layer is the initial conversion phase between the input data and the transformer. The input, $\mathbf{X} = [x_1, x_2, \dots, x_N]$, is transformed into the token-embedding matrix $\mathbf{Z} \in \mathbb{R}^{N \times d_{\text{model}}}$, where N is the number of decimal integer values in the sequence and d_{model} is the dimension of the embedding vectors. The embedding layer converts these input sequences, encoded and padded during the preprocessing phase, into embedding vectors of fixed size 768. This conversion is performed by the learnable embedding matrix $\mathbf{W}_E$, which maps the fixed input tokens to their corresponding embedding vectors in the feature space. The output, $\mathbf{Z}$, is given by:

$$\mathbf{Z} = \mathbf{W}_E \mathbf{X}, \quad (1)$$

where $\mathbf{W}_E$ is the embedding matrix, with each distinct $\mathbf{W}_E$ corresponding to a particular instantiation of the embedding parameters that will be learned during training.

2) POSITIONAL ENCODINGS

For transformers to understand and preserve the order of sequences, positional encodings are added to the input embeddings of the model. These encodings provide a unique position to each element in the sequence, allowing the model to recognize the position of each byte in the network packet and their relation to others within the same sequence. This aids the self-attention mechanism by treating each input independently and would otherwise not maintain information about sequence order.

The positional encoding vector is designed to have the same dimension, d_{model} , as the embedding layer, allowing the

two to be summed together. The encoding for position pos and dimension i is calculated using sine and cosine functions:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (3)$$

This ability lets the model shift its attention globally across the sequence, paying more attention to certain parts based on what they are and their position. This makes the model more efficient at recognizing patterns within the sequence.

3) MULTI-HEAD SELF-ATTENTION MECHANISM

The self-attention mechanism is what sets transformers apart from other traditional deep learning architectures. The self-attention mechanism allows the model to capture dependencies and relationships across the entire input sequence, regardless of their distance from each other. The ability to weigh the importance of different bytes based on both their content and their positions within the sequence is what makes the self-attention mechanism powerful for the given sequential task. This is achieved through the following steps:

4) INPUT LINEAR TRANSFORMATION

Once the input sequence has been converted into the embedded matrix $\mathbf{Z}$, each vector $\mathbf{z}_i$ within this matrix is then transformed into three distinct vectors: queries ($\mathbf{Q}$), keys ($\mathbf{K}$), and values ($\mathbf{V}$), using the respective weight matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$, which are learned during the training process. These transformations can be described by the following equations:

$$\mathbf{Q} = \mathbf{W}_Q \mathbf{Z} \quad (4)$$

$$\mathbf{K} = \mathbf{W}_K \mathbf{Z} \quad (5)$$

$$\mathbf{V} = \mathbf{W}_V \mathbf{Z} \quad (6)$$

where $\mathbf{Z}$ is the matrix of embedded vectors, and $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are the weight matrices that transform the embedded vectors into queries, keys, and values, respectively.

5) COMPUTATION OF ATTENTION SCORES

Attention scores are computed by taking the dot product of the query with all keys, scaled by $\frac{1}{\sqrt{d_k}}$, where d_k is the dimension of the key vectors. The softmax function is applied to these scores to obtain a probability distribution.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (7)$$

The final output represents an aggregation of the input features, detailing the most relevant information for each byte in the sequence. This mechanism allows the model to capture both the global understanding and complex information necessary for understanding the packet input patterns.

6) MASKED PREDICTION LAYER

The primary function of the masked prediction layer is to predict the masked bytes in the input sequence. This layer

Algorithm 1 Self-Supervised Training and Embedding Extraction

- 1: **Input:** Set of network packets $P = \{p_1, p_2, \dots, p_n\}$
- 2: **Output:** Packet embeddings $\mathbf{E}$ for downstream tasks
- 3: **Step 1: Packet Byte Masking**
- 4: **for** each packet $p_i \in P$ **do**
- 5: Convert p_i to decimal integers $\mathbf{x} = (x_1, x_2, \dots, x_l)$, where $x_j \in [0, 255]$.
- 6: Let $\alpha = 0.15$. Randomly select $\lceil \alpha \cdot l \rceil$ positions from $\{1, 2, \dots, l\}$ to form the mask set R .
- 7: **for** each index $j \in R$ **do**
- 8: With 80% probability: $x_j \leftarrow [\text{MASK}]$
- 9: With 10% probability: $x_j \leftarrow \text{Random token} \in [0, 255]$
- 10: With 10% probability: $x_j \leftarrow \text{Unchanged}$
- 11: **end for**
- 12: Pad $\mathbf{x}$ to fixed length L_{max} to form p'_i .
- 13: **end for**
- 14: Let $P' = \{p'_1, p'_2, \dots, p'_n\}$ be the masked dataset.
- 15: **Step 2: Transformer Model Training**
- 16: For each $p'_i \in P'$, embed via $\mathbf{W}_E$ to get $\mathbf{Z}_i$. Add positional encodings $\mathbf{PE}$.
- 17: Train the transformer using self-attention to predict masked bytes:

$$\min_{\theta} \mathcal{L}(\theta),$$

where $\mathcal{L}$ is the cross-entropy over masked positions. Use Adam for optimization.

18: Step 3: Embedding Extraction

- 19: After convergence freeze transformer's parameters θ .
- 20: For any input packet p_i , feed forward to obtain last hidden state embeddings $\mathbf{H}_i$.
- 21: Compute the packet embedding $\mathbf{e}_i$ by mean pooling the rows of $\mathbf{H}_i$.
- 22: **Return** the set of packet embeddings $\{\mathbf{e}_i\}$.

converts the encoded embeddings into logits corresponding to the vocabulary size. These logits are then passed through a softmax function to produce a probability distribution over the entire vocabulary for each masked position. These probabilities represent the model's predictions for the masked bytes in the input sequence.

B. MODEL TRAINING

The self-supervised transformer model is trained on packet data without labels, leveraging a MLM objective at the byte-level. Following standard masking strategies [5], 15% of bytes in each input sequence are randomly selected for masking. Out of those selected for masking, 80% are replaced with a special mask token, 10% are replaced by a random byte sampled from the vocabulary, and the remaining 10% are left unchanged. Although some selected bytes are not physically replaced by the mask token, they are still considered "masked" from the model's perspective. Therefore, the model learns to predict the original values of these masked

Algorithm 2 Few-Shot Learning for Malware Classification1: **Input:**

- Support set S containing examples from three malware classes (one of which is unseen/untrained).
- Query set Q (unlabeled samples for evaluation).

2: **Output:** Predicted labels $\hat{y}_q$ for each query point $q \in Q$.3: **Step 1: Episodic Training**4: **for** training episode = 1 **to** max_episodes **do**5: Randomly generate a support set S_k and query set Q_k for each class $k = 1, \dots, K$.6: **for** each class $k \in \{1, \dots, K\}$ **do**

7: Compute the prototype:

$$c_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\theta(x_i).$$

8: **end for**9: **for** each query sample $(x_j, y_j) \in Q_k$ **do**10: Compute embedding: $\mathbf{e}_j = f_\theta(x_j)$ 11: Assign predicted label: $\hat{y}_j = \arg \min_k d(\mathbf{e}_j, c_k)$ 12: **end for**13: Compute the loss J over the query predictions.14: **end for**15: **Step 2: Few-Shot Inference**16: Compute prototypes c_k for each class k using the support set S .17: **for** each query sample $q \in Q$ **do**18: Compute embedding: $\mathbf{e}_q = f_\theta(q)$ 19: Assign predicted label: $\hat{y}_q = \arg \min_k d(\mathbf{e}_q, c_k)$ 20: **end for**

bytes using the context provided by the unmasked bytes. Cross-entropy loss is employed as the primary cost function for masked byte prediction. The cross-entropy can be defined as:

$$\mathcal{L} = -\frac{1}{N_{\text{masked}}} \sum_{i=1}^N \sum_{c=1}^V y_{i,c} \log(p_{i,c}), \quad (8)$$

where V is the number of unique packet elements, $y_{i,c}$ represents the ground truth for the i -th masked element, and $p_{i,c}$ is the predicted probability for each element c at position i . N is the total count of masked elements across the batch, and N_{masked} is used to normalize the loss. This approach focuses the model on predicting masked elements using the context provided by the unmasked elements, leading to learn the representation of the data's underlying structures.

The Adam optimizer is used with a learning rate of $2e - 5$ and provides weight decay regularization, which prevents the weights from growing too large to help mitigate overfitting [39]. Overfitting occurs when a model fits too closely to the training dataset, leading to poor performance on the testing data. A scheduler for learning rate decay is also used to reduce the learning rate over the training period. The scheduler gradually increases the learning rate from zero to a specified learning rate during the warmup period,

then linearly decreases the learning rate over the remaining epochs of training [40]. The experiments were conducted on NVIDIA GeForce RTX 2080 GPUs. This hardware enabled us to use significant computational power, facilitating quicker processing and learning.

C. EMBEDDING EXTRACTION

After the training is complete, we process each input packet sequence through the trained transformer's final hidden layer to produce a contextualized embedding vector for every byte in the input sequence. To obtain a vector representation for each packet, we perform mean pooling over the sequence dimension of the last hidden states. Let $\mathbf{H} \in \mathbb{R}^{N \times d_{\text{model}}}$ represent the last hidden state, where N is the sequence length and d_{model} is the embedding dimension. We compute the final packet embedding $\mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$ as:

$$\mathbf{e} = \frac{1}{N} \sum_{i=1}^N \mathbf{H}_i. \quad (9)$$

This operation aggregates the information across all bytes in the sequence, resulting in a single embedding vector that captures the semantic features of each packet. These embeddings are used as input for downstream tasks, such as the fully-connected classifiers. For the FSL task, when a novel class is introduced that is excluded from SSL pre-training, its packet sequences are still passed through this same final hidden layer to extract embeddings. This enables the transformer's learned feature space to generalize to novel or unseen classes. Algorithm 1 details the masking, training, and extraction of the SSL model.

D. MODEL CONFIGURATION

The selected hyperparameters of the model include the number of unique bytes (*i.e.* 258), hidden size, number of hidden layers, number of attention heads, the intermediate size, maximum length of the input payload, and the number of labels for this application, and are presented in Table 2.

The unique bytes parameter represents the total number of unique elements found in the input data. Here, the total is 258, accounting for each unique hexadecimal value and additional pad, unknown, and masking integers, following standard self-supervised practices. The hidden size parameter is essential in determining the dimension of the hidden state. The number of hidden layers represents the number of stacked transformer blocks present in the model. Complementing these blocks is the number of attention heads, which control the model's ability to attend to different segments of the input. This is pivotal for the model to comprehend and learn from the input data effectively. The intermediate size determines the dimension of the intermediate later in the network within each transformer layer. This value helps ensure that the network can process and transform the input data at each layer effectively. The maximum position embeddings denote the maximum length of the input sequence. For this study, we do not restrict the maximum packet length, which is set

TABLE 2. Model configurations.

Configuration Parameter	Value
unique_bytes	258
hidden_size	768
num_hidden_layers	12
num_attention_heads	12
intermediate_size	3072
max_position_embeddings	1500

TABLE 3. Downstream M.L. classifier architecture.

Layer	Output dim.	Components
FC: $d_{\text{model}} \rightarrow 512$	512	ReLU, BatchNorm, Dropout(0.5)
FC: $512 \rightarrow 128$	128	ReLU, BatchNorm
FC: $128 \rightarrow C$	C	softmax

to 1500. It is important to note that transformer architectures are inherently capable of processing sequences that extend beyond a single packet's MTU, meaning that if a payload exceeds 1460 bytes and is split across multiple packets, the transformer can process it as a continuous stream of bytes across packet boundaries. However, increasing the maximum input length significantly raises computational resource requirements, such as memory usage and processing time. These configurations remained consistent through our study of extracting the packet byte embeddings.

E. DOWNSTREAM TASK

After the extraction of the embeddings from the transformer trained with SSL approaches, a downstream task is applied for the purpose of malware classification to test the quality of the embeddings. In this paper, we assume *two downstream task settings*: 1) sufficient annotated malware samples are available, enabling the training of a conventional classifier; and 2) only a few annotated samples are available, leading to the implementation of FSL.

In the first setting, we assume that a sufficient amount of labeled data is available. This is tasked with interpreting and classifying the embeddings to distinguish between benign and malicious network traffic and classifying the type of malwares. To achieve this, a **multi-layer (M.L.)** classifier architecture is employed on top of the transformer model.

In this architecture, shown in Table 3, the input embedding vector of dimension d_{model} is first passed through a fully connected linear layer that projects it into a 512-dimensional space. This is followed by batch normalization, a ReLU activation to introduce non-linearity, and a dropout layer with a dropout rate of 0.5 to prevent overfitting. The network then reduces the 512-dimensional output to 128 dimensions through a second linear layer, again applying batch normalization and a ReLU activation to maintain stability and non-linearity in the learning process. Finally, a linear layer maps the 128-dimensional vector to C output neurons, where C corresponds to the number of classes. A softmax activation function is applied to the final output to generate a probability distribution over the classes.

The model is trained using cross-entropy loss and the Adam [39] optimizer is implemented with a learning rate of 0.01. A learning rate scheduler is also incorporated into training, ensuring the model adapts its learning process over time. After training, the classifier interprets the embeddings to distinguish between benign and malicious packets. The classifier outputs a probability distribution across all defined categories, such as benign or malicious for binary, and types of malware for multiclass.

VII. FEW-SHOT LEARNING ARCHITECTURE

Our framework begins by pre-training the transformer model using SSL to extract the robust embeddings from network traffic data. When sufficient labeled data exists, these embeddings are used to train a downstream classifier for malware detection and classification. This classifier is highly effective for known malware types when there exists a large number of annotated data samples. However, when a new malware variant occurs in a network, often only a few labeled examples are available for these novel threats. In such cases, it is impractical to train a new classifier from scratch due to the scarcity of data. To address this challenge, we employ a FSL approach that leverages robust representations learned during pre-training on known malware types. These representations enable the adaptation to novel malware classes with only a few annotated examples by transferring prior knowledge to classify new threats.

In the FSL scenario (the second setting), once the embeddings of the packets are extracted using the pre-trained transformer, the support and query sets are randomly selected. Support sets consists of a limited number of labeled examples from each class, while the query set contains unlabeled examples that the model attempts to classify based on the knowledge learned from the support set. The model uses a limited number of support examples (shots) from each class (ways) to form class-specific prototypes. These prototypes represent the central tendencies of each class in the embedding space from the self-supervised features. The few-shot framework employs a fully connected linear layer, tasked with transforming the input embeddings to a representation space. The model is also trained using episodic training. Episodic training is a method used in few-shot learning to prepare models by organizing training into a series of learning problems, known as episodes, to mimic the circumstances encountered during testing [41].

In each episode, we randomly select K classes from the dataset. For each class k , two distinct sets are sampled: a support set S_k comprising N_S examples, and a query set Q_k comprising N_Q examples. The algorithm computes the class prototypes $\mathbf{c}_k$ by taking the mean of the transformed feature vectors of the support set embeddings, S_k . Once the class prototypes are established, the algorithm evaluates each query sample (x, y) in Q_k , calculating the Euclidean distance from x to each class prototype $\mathbf{c}_k$, and then assigns the class label of the nearest prototype to x . The update to the model parameters is based on the loss J , which is formulated as the

TABLE 4. Results of binary and multiclass classification of malware types on the test dataset.**Note:** Bold font indicates the highest performance, while underlined denotes the second-best result in each column.

Method	Binary Classification								Multiclass Classification							
	UNSW-NB15				CIC-IoT23				UNSW-NB15				CIC-IoT23			
	Acc.	Prec.	Rec.	F1.	Acc.	Prec.	Rec.	F1.	Acc.	Prec.	Rec.	F1.	Acc.	Prec.	Rec.	F1.
1D-CNN [21]	95.19	94.89	95.77	96.31	<u>82.81</u>	<u>82.33</u>	<u>82.51</u>	<u>82.92</u>	89.88	90.15	89.88	89.88	<u>76.18</u>	77.99	<u>76.19</u>	<u>76.37</u>
2D-CNN [22]	92.95	92.91	92.91	92.91	78.89	79.26	78.89	78.80	87.73	88.55	87.73	87.75	<u>69.81</u>	76.48	<u>69.81</u>	70.02
LSTM [42]	91.47	90.41	93.25	91.81	79.22	78.96	80.31	79.63	83.26	83.98	83.26	83.42	61.47	61.97	60.89	61.45
ResNeXt+ [14]	91.66	92.11	91.16	91.13	82.10	82.11	82.10	82.09	<u>90.05</u>	<u>90.18</u>	<u>90.05</u>	<u>90.12</u>	75.14	<u>79.14</u>	75.14	75.29
DEEPShield [13]	92.77	89.99	96.66	93.20	75.56	73.23	81.44	77.12	84.84	85.78	84.84	84.79	69.41	73.69	69.41	69.52
Proposed (M.L.)	<u>94.88</u>	95.04	94.88	<u>94.88</u>	83.25	83.25	83.25	83.24	91.29	91.51	91.29	91.29	77.88	79.55	77.88	78.03

TABLE 5. Average inference time per sample (ms).

Method	UNSW-NB15		CIC-IOT23	
	Binary	Multiclass	Binary	Multiclass
1D-CNN [21]	0.51	0.16	0.13	0.14
2D-CNN [22]	0.61	0.64	0.60	0.59
LSTM [42]	6.10	6.33	5.80	5.63
ResNeXt+ [14]	1.98	1.71	1.93	1.95
DEEPShield [13]	0.34	0.40	0.28	0.29
Proposed (M.L.)	0.03	0.06	0.04	0.04

negative log-likelihood of the softmax probabilities over the distances between the query samples and the class prototypes. Specifically, for each query (x, y) in Q_k , the loss J is updated according to the equation:

$$J \leftarrow J + \frac{1}{KN_Q} \left[d(f_\theta(x), c_k) + \log \sum_{k'} \exp(-d(f_\theta(x), c_{k'})) \right]. \quad (10)$$

This process is repeated across numerous episodes, allowing the model to generalize better to new configurations of class examples. The overall procedure of the proposed framework is highlighted in Algorithm 2.

VIII. RESULTS

In this section, we delve into the metrics to report performance, the state-of-the-art (SOA) benchmarks used for comparison, and the results of our proposed method. We pre-train the encoder once per experimental setting and then run each downstream classifier three times with different random seeds, reporting the mean test accuracy (and episode-averaged mean for FSL).

A. METRICS

To evaluate the performance of the proposed method, several metrics are taken into consideration. For this study, we consider accuracy, precision, recall, and F1-Score on the test dataset. True positives (TP) are instances when the model correctly identifies a packet as malicious, a true negative (TN) are instances where the model correctly identifies the packet as benign, false positives (FP) are instances where the model incorrectly identifies a benign packet as malicious, and

false negatives (FN) when the model mistakenly identifies a malicious packet as benign. The accuracy is interpreted as the proportion of packets that the model correctly identifies, as either malicious or benign, out of all sampled packets, shown by $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$. Precision measures the proportion of packets identified by the model as malicious that are truly malicious, $Precision = \frac{TP}{TP+FP}$. Recall evaluates the proportion of all malicious packets present that the model successfully identifies, $Recall = \frac{TP}{TP+FN}$. The F1-Score is the harmonic mean of precision and recall, $F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall}$. These metrics provide a comprehensive evaluation of our model's ability in accurately detecting and classifying malware. Lastly, for clarification, the test dataset used in our evaluation process differs from the training dataset. The test dataset consists of randomly selected, unseen samples and ensures that the model is using a distinct dataset to evaluate the performance.

B. BENCHMARKS

Central to the evaluation of our study is the role of the downstream classifier, which confirm the effectiveness of the transformer embeddings produced through SSL methods. To display our proposed method's robustness, we compare with other SOA learning models for traffic classification that are comparable to our study. The 1D-CNN model [21] employs one-dimensional convolutional layers to extract hierarchical patterns from raw network packet data, effectively capturing local dependencies within sequential data. The 2D-CNN model [22] extends this concept by mapping packet data into a two-dimensional grid structure, allowing it to capture spatial relationships and patterns that may not be evident in a sequential representation. The LSTM [42] aids in modeling long-term dependencies in sequential data, making it proficient for capturing temporal patterns in packet streams. ResNeXt+ [14] builds on the ResNeXt architecture by incorporating an attention mechanism into the residual blocks for feature exaction. Lastly, DEEPShield [13] takes a hybrid approach, developing an ensemble method technique that includes a CNN and LSTM. All these models are evaluated using the same SSL-generated embeddings, ensuring a consistent feature representation across comparisons and showcasing the adaptability of our embeddings to diverse architectures.

TABLE 6. Results of few-shot classification of malware types on the query set.

Dataset	Trained Classes	Unseen Class	1-shot			5-shot		
			Acc.	F1	Unseen Acc.	Acc.	F1	Unseen Acc.
UNSW-NB15	Exploits, Fuzzers	Generic	83.04	82.39	82.82	89.12	89.10	89.95
	Fuzzers, Generic	Exploits	82.86	82.26	82.54	89.48	89.74	89.68
	Exploits, Generic	Fuzzers	79.42	78.59	77.45	87.89	87.65	87.16
CIC-IoT23	Backdoor, Vulnerability	Brute Force	58.23	57.19	56.08	71.37	71.08	70.67
	Vulnerability, Brute Force	Backdoor	61.32	60.25	61.35	74.26	73.89	74.07
	Backdoor, Brute Force	Vulnerability	62.35	61.38	61.73	75.01	74.91	75.05

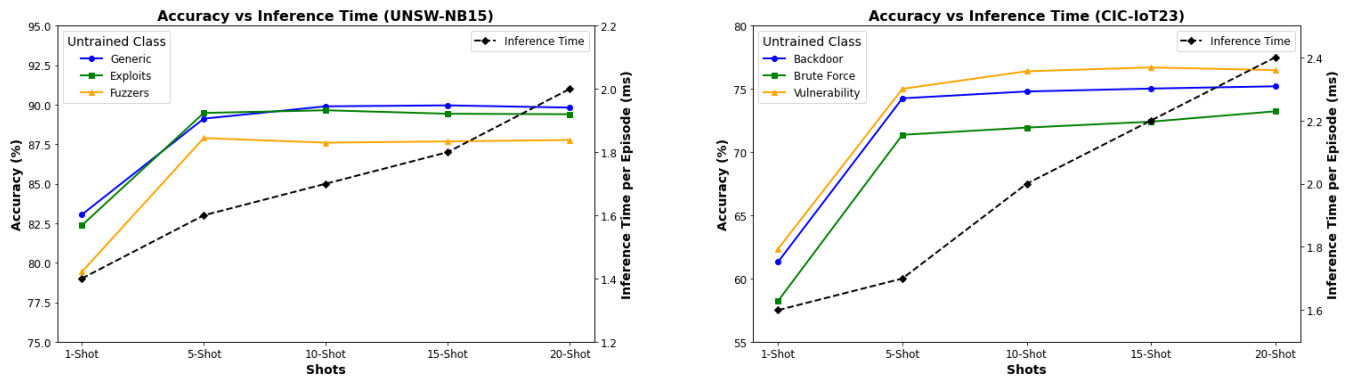


FIGURE 4. Comparison of accuracy and inference time UNSW-NB15 and CIC-IoT23 Datasets: Accuracy trends for different untrained classes are displayed alongside average inference time per episode for each number of shots.

C. BINARY AND MULTI-CLASS CLASSIFICATION

The data for the downstream classification task involving the SSL embeddings are divided into training, testing, and validation sets, with proportions of 70%, 20%, and 10%, respectively. The performance of our proposed method was evaluated over 25 epochs, demonstrating improvements in accuracy, precision, recall, and F1-Score when using packet bytes as input, shown in Table 4.

For the binary classification task, the M.L. fully connected network demonstrates strong performance, with the M.L. classifier achieving SOA results. For the UNSW-NB15 dataset, the M.L classifier attains an accuracy of 94.76%, precision of 94.95%, and an F1-score of 94.81%. Similarly, on the CIC-IOT23 dataset, the M.L classifier outperforms all other method's metrics, achieving an accuracy of 83.25% and consistently high precision and recall scores. Similarly, when extending to a multiclass classification task to distinguish different malware types, a similar trend occurs. For example, the M.L. model achieves 91.51% accuracy classifying Fuzzers, Exploits, and Generic attacks, and 78.03% F1-score on CIC-IoT23 for classifying Backdoor, Vulnerability, and Brute Force attacks. Although the simpler M.L. model exhibits SOA performance, we also observe that other downstream architectures (i.e. CNN or DEEPShield) exhibit strong performance with the produced SSL embeddings. These findings confirm that the SSL transformer's embeddings preserve valuable information about packet data,

enabling robust performance across a range of classifier architectures.

The inference times per sample, shown in Table 5, further emphasize the efficiency of the proposed method. The M.L. fully connected networks has a significantly lower inference time compared to other benchmark models. The M.L network achieves an inference time of 0.03 ms and 0.04 ms for UNSW-NB15 and CIC-IOT23, respectively. In contrast, models such as LSTM and CNNs exhibit considerably higher inference times, with the LSTM model requiring over 6 ms per sample on UNSW-NB15. The faster inference time of the M.L. network make it ideal for real-time applications.

The enhancement in metrics can be attributed to several key factors inherent to our model's design and training approach. The extraction of the SSL embeddings allows our model to capture a deeper understanding of the dataset's intrinsic structure. Traditional supervised learning rely on labeled data, while SSL focuses on learning useful representations from unlabeled data via a pretext task, enabling the model to better generalize for the training and testing of the data. The improvement in metrics is also due to the transformer-based approach, which handles sequential data more efficiently due to the self-attention mechanism. This approach allows the model to evaluate each byte based on position within the entire sequence, giving it a unique advantage in identifying the significance of each byte relative to the whole sequence. The ability to globally understand the sequence helps identify

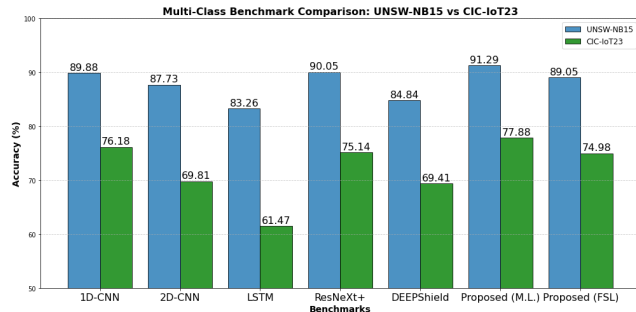


FIGURE 5. Comparison of accuracies of benchmarks vs. few-shot classification.

when certain byte sequences or patterns might be indicative of malicious activity when appearing in specific contexts or configurations within the packet.

D. FEW-SHOT CLASSIFICATION

1) EXPERIMENTAL SETUP

The primary motivation for these experiments is to assess the model's ability to generalize to novel classes with limited supervision, a critical requirement for real-world scenarios where labeled data for all classes may not always be available. FSL allows us to test how well the SSL embeddings can adapt to previously unseen classes with very few labeled examples. To simulate this scenario, we train the SSL model exclusively on **two of the three malware classes** during pre-training, ensuring no prior exposure to the unseen class.

We frame the problem as a 3-way classification task, testing both 1-shot and 5-shot settings, and using 15 query samples per episode. During each training episode, the model is presented with a small support set (either 1 or 5 labeled examples per class) and a set of query samples drawn from all three classes, including both previously seen (trained) and novel (untrained) classes. By exposing the model to queries from both known and new classes within the same episode, we ensure that it develops a global understanding of the entire class distribution, rather than focusing solely on the introduced class. Furthermore, we report the accuracy for the unseen class during inference.

The training process spans 10 epochs, with 500 episodes per epoch for the UNSW-NB15 dataset and 2,000 episodes per epoch for the CIC-IoT23 dataset, reflecting their respective sizes (over 8,000 and 40,000 samples). Following training, the model is evaluated using 1,000 newly sampled episodes, structured similarly to the training episodes. To maintain a fair evaluation, training and testing episodes are drawn from disjoint subsets of the dataset. To further validate the robustness of our approach, we repeat the entire training and evaluation process over five independent iterations. For each iteration, the model's weights are re-initialized, and the training procedure (including all epochs and episodes) is executed from scratch.

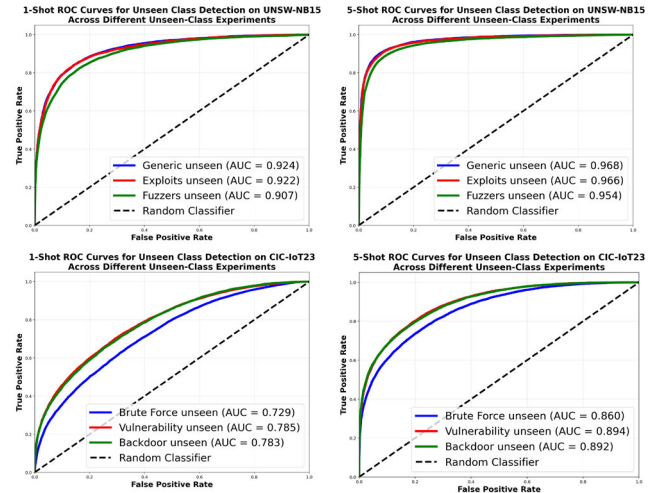


FIGURE 6. Few-Shot ROC for Unseen Class on UNSW-NB15 and CIC-IoT23, 1-/5-shot.

2) RESULTS

The results of our few-shot learning experiments for classifying both known and newly introduced malware types are presented in Table 6. The *Trained Classes* column indicates which classes were pre-trained with self-supervision, with the *Unseen Class* column serving as the novel, untrained class for that specific experiment. In the 1-shot setting on the UNSW-NB15 dataset, the model's highest performance occurs when trained on Exploits and Generic malware types, with an overall query accuracy of 83.04% and 82.82% accuracy specifically for the unseen class. In the 5-shot setting, the best overall query accuracy of 89.48% is achieved when the model is trained on Fuzzers and Generic malware types, with Exploits as the unseen class. Meanwhile, the highest accuracy for an unseen class is observed for the Generic class, achieving 89.95%. For the CIC-IoT23 dataset, the model demonstrates a similar trend but with slightly lower accuracies, reflecting the increased diversity of the IoT traffic data. In the 1-shot setting, the best performance is achieved when training on Backdoor and Brute Force malware types, with Vulnerability as the unseen class, resulting in an overall query accuracy of 62.35% and 61.73% accuracy for the unseen class. Similarly, in the 5-shot setting, the model achieves its highest query accuracy of 75.01% when trained on Backdoor and Brute Force malware types, with Vulnerability as the unseen class.

Fig. 4 offers additional context by comparing the relationship between accuracy and inference time for both datasets across different shot settings. The most significant accuracy improvement is observed between the 1-shot and 5-shot scenarios, with smaller incremental gains as the number of shots increases. The increase in accuracy between 1 and 5 shots shows how providing more support examples leads to the generation of more discriminative class prototypes. Specifically, on the UNSW-NB15 dataset, the model demonstrates a steady increase in accuracy with

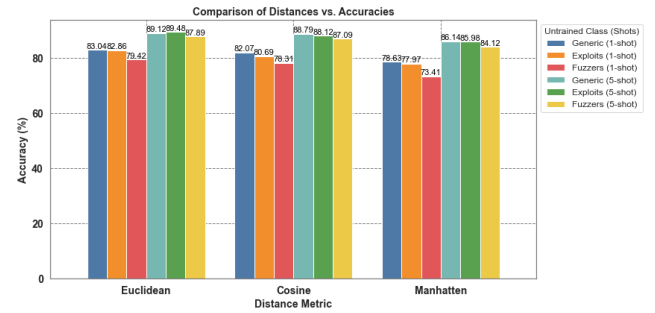
TABLE 7. Masking percentage ablation with M.L. classifier (binary).

Masking %	UNSW-NB15		CIC-IoT23	
	Accuracy	F1-Score	Accuracy	F1-Score
5%	95.59 ± 0.22	95.59 ± 0.23	79.53 ± 0.33	79.48 ± 0.31
10%	95.47 ± 0.60	95.47 ± 0.60	80.69 ± 0.37	80.68 ± 0.37
15%	94.88 ± 0.15	94.88 ± 0.15	83.25 ± 0.25	82.34 ± 0.24
20%	94.28 ± 0.31	94.28 ± 0.31	83.17 ± 0.20	83.15 ± 0.20
25%	94.57 ± 0.23	94.57 ± 0.23	85.27 ± 0.21	85.28 ± 0.21
30%	93.97 ± 0.36	93.97 ± 0.36	84.93 ± 0.28	84.92 ± 0.28

more shots before plateauing, showing optimal performance with minimal inference time. For example, the Generic class reaches approximately 90% accuracy in the 15-shot setting, with an inference time of just over 2ms per episode. Similarly, in the CIC-IoT23 dataset, accuracy trends improve with more shots, with Vulnerability reaching over 76% accuracy in the 15-shot setting. The inference times remain low for both datasets, demonstrating the computational efficiency of the proposed approach. Furthermore, it is important to note that an episode in our few-shot learning framework is not a single sample but consists of multiple samples: a support set containing N_S labeled examples per class and a query set containing N_Q unlabeled examples from all classes. For a 3-way classification task, each episode therefore includes $3 \times (N_S + N_Q)$ samples.

Furthermore, we opt to compare the average few-shot results with the benchmarks to analyze how the accuracy compares, shown in Fig. 5. Despite using significantly fewer training samples, the FSL method performs comparably to SOA classifiers, surpassing methods like 2D-CNN, LSTM, and DEEPSHIELD classifiers. Additionally, in our FSL framework, we train on only two of three classes, unlike the previously mentioned multiclass classification problem, which required all three classes to undergo self-supervised training. This in-turn saves training costs and allows for the ability to detect previously unseen malware classes with high accuracy, an issue that is overlooked in typical classification problems. Overall, our approach effectively leverages knowledge from known malware types to classify novel malware types that were unseen during training, even with a limited number of labeled samples. This capability makes it a promising solution for real-world scenarios where labeled data is scarce and new malware types are frequent.

Lastly, as shown in Fig. 6, we present the Receiver Operating Characteristic (ROC) curves corresponding to the unseen-class few-shot experiments. Each curve demonstrates model performance in both the 1-shot and 5-shot settings, reported separately for the UNSW-NB15 and CIC-IoT23 datasets. By plotting the true positive rate against the false positive rate, these curves highlight the model's ability to distinguish unseen classes. For UNSW-NB15, all unseen classes achieve strong separability, with AUC values above 0.90 in both 1-shot and 5-shot settings. This indicates that even with minimal labeled support data, the learned embeddings

**FIGURE 7. Comparison of average few-shot classification accuracies across different distance metrics with the UNSW-NB15 dataset.**

provide highly discriminative features, and performance further improves with additional support examples. For CIC-IoT23, unseen detection is more challenging, as reflected in lower AUC values (0.73–0.78) in the 1-shot setting. However, the curves show improvement with 5-shots, with AUC values improving above 0.86 across all unseen categories. This demonstrates that the model generalizes better when provided with slightly more support samples, particularly in IoT traffic where class distributions and payload patterns are more diverse.

IX. ABLATION STUDY

A. MASKING PERCENTAGE

To investigate how varying proportion of masked bytes influences the quality of the learned embeddings, we conduct an ablation study examining multiple masking percentages ranging from 5% to 30%. Although 15% masking is generally used for SSL tasks, our objective is to learn the optimal ratio that preserves sufficient information for robust training of the model, but also prevents the model from becoming too reliant on unmasked data. Table 7 displays the results for different masking percentages on distinguishing between malicious and benign packets. On the UNSW-NB15 dataset, we can observe the highest accuracies occur between 5% and 15%, with accuracy slightly dropping off when masking more than 15% of packet bytes. On the more challenging CIC-IoT23 dataset, we observe that higher masking rates improve performance, with 25% byte masking resulting in over a 2% increase in classification over standard 15% masking.

The results suggest that the conventional 15% of input masking commonly employed in SSL tasks may not be universally optimal for all network traffic domains. Instead, choosing an appropriate masking rate depends on dataset complexity and diversity. On more homogeneous network traffic, such as UNSW-NB15, a moderate masking strategy works well. However, on the CIC-IoT23 dataset, more varied patterns emerge due to complexity of the number of devices, over 100, connected on the same network. These additional complexities benefit from a higher masking ratio, which forces the model to rely more heavily on context rather than a smaller subset of repetitive byte patterns.

TABLE 8. Transformer depth and dimension ablation on UNSW-NB15 for binary, multiclass, and FSL classification.

Depth / Dim	Binary		Multiclass		FSL (5-shot)	
	Acc.	F1	Acc.	F1	Acc.	Unseen Acc.
6 / 384	76.03	75.06	85.46	85.45	68.26	67.52
6 / 768	92.57	92.57	89.31	89.31	82.88	82.60
12 / 384	82.81	82.60	84.73	84.68	70.19	69.42
12 / 768 (baseline)	94.88	94.88	91.29	92.29	88.83	88.93

B. FEW-SHOT DISTANCE METRIC

To evaluate the impact on various distance metrics on few-shot classification, we experiment with three common measures: Euclidean distance, Cosine similarity, and Manhattan distance. Euclidean distance is used to measure the straight-line distance between the class prototype and the query point to be classified, cosine similarity is the cosine of the angle between two vectors and the dot product of the vectors normalized by the product of their lengths, and lastly Manhattan distance is computed as the sum of the absolute differences between coordinate values. Euclidean distance was exclusively implemented in our main experiments. By only changing the distance metric while keeping all other hyperparameters fixed, we can isolate the effect of the distance function on few-shot performance.

The results of the ablation study are shown in Fig. 7 and demonstrate notable variations in classification performance across all three distance metrics. Notably, Euclidean distance consistently outperforms Cosine and Manhattan distances on all three few-shot experiments with the UNSW-NB15 dataset, resulting in an average accuracy of $81.77\% \pm 2.04\%$ and $88.83\% \pm 0.83\%$ in the 1 and 5-shot settings, respectively. Cosine distance, although slightly less effective than Euclidean distance, still produces competitive results, achieving an average accuracy of $80.36\% \pm 1.90\%$ in the 1-shot setting and $88.00\% \pm 0.86\%$ in the 5-shot setting. Lastly, Manhattan distance demonstrates the weakest performance among the three distances, with an average accuracy of $76.67\% \pm 2.84\%$ and $85.41\% \pm 1.12\%$ in the 1 and 5-shot settings.

C. SENSITIVITY TO TRANSFORMER PARAMETERS

To investigate the sensitivity to transformer depth and embedding dimension, we conduct an ablation study to evaluate the sensitivity of our model to the depth of the transformer backbone and the dimensionality of its embeddings. Our original architecture is based on a 12-layer transformer with a hidden size of 768, consistent with standard base configurations. To simulate resource-constrained settings for SSL pre-training, we designed variants with reduced numbers of transformer layers and hidden sizes.

The rationale behind this design consists of two main reasons. First, the number of transformer layers in the model influences the representational capacity of the model, while also corresponding to the number of attention heads

the model can apply. This enables deeper networks to capture longer-range dependencies and better refine attention patterns, although at the cost of increased computational overhead during pre-training. Second, the size of the embedding dimension determines how much information is retained in the latent space. Smaller dimensions may improve computational efficiency, but lose fine-grained and discriminative characteristics of the traffic bytes. By evaluating different configurations, we can better understand the relationships between model depth, representational power, and classification performance across binary, multiclass, and FSL scenarios.

Table 8 summarizes the results of this ablation on the UNSW-NB15 dataset, compared to baseline classification performance of each experiment. We report accuracy and F1 metrics across binary and multiclass experiments, and overall and unseen class accuracies for the FSL experiment based on the 5-shot baseline results, averaged over all three FSL experiments. Reducing transformer layer depth from 12 to 6 layers while retaining the 768-dimensional embedding yields the smallest degradation across all tasks. In contrast, narrowing the embedding to 384 dimensions across either depth substantially decreases performance, with the largest drops occurring in the binary and FSL classification tasks. The baseline models remains best overall, but among the reduced-capacity models, we can conclude that the 6/768 variant offers a favorable trade-off.

This occurs due to the smaller embedding dimension constraining the model's capacity to encode discriminative features from the traffic data, especially under few-shot scenarios, where the model must adapt to the novel class that was not seen during pre-training. Consequently, this leads to the largest performance drop in the few-shot setting for average and unseen accuracies. Regarding the depth of the layers, we can conclude that most discriminative features are learned in the earlier layers of the model. However, the baseline performance of 12/768 still yields favorable results compared to the constrained variants.

X. ENCRYPTED TRAFFIC

In this study, our analysis focuses on the raw packet bytes contained within TCP and UDP network packets from various datasets. However, it is crucial to note the limitations when dealing with encrypted traffic. Encryption—the process of converting plaintext into ciphertext to secure data transmission and make it unreadable to unauthorized entities without the appropriate decryption keys—plays a pivotal role in data security. Specifically, when plaintext is encrypted using a key, it transforms into ciphertext that does not reveal any information about the original content without the correct decryption key [43].

Let us denote m as the variable representing the raw payload message, and $E_k(\cdot)$ as the encryption function that encrypts m with the key k . In this context, observing the ciphertext c yields no information about the original

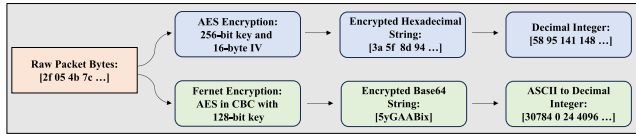


FIGURE 8. Comparison of processing input packets for AES and Fernet encryption processes.

message m , expressed as:

$$p(m = m_1 | E_k(m) = c) = p \quad (11)$$

This equation underscores that decrypting the plaintext m_1 with the encryption algorithm $E_k(\cdot)$ and key k results in a unique ciphertext, effectively concealing any details of the plaintext.

Despite these encryption properties, some studies have successfully classified encrypted network traffic from various applications [44], [45]. This classification success is largely due to the unique encryption signatures that different applications' random generators leave, which, while not disclosing plaintext content, offer a form of application-specific pattern. The challenge, however, lies in malware detection, where access to plaintext information is crucial. If malware signatures can be detected in encrypted traffic, it would suggest that the encryption method fails to adequately mask the plaintext, thereby violating the principle stated in Equation 11. Our research involved applying two encryption methods to the packet data: the Advanced Encryption Standard (AES) and Fernet symmetric encryption [46], [47]. Unlike our earlier work [20], which introduced text-encoding artifacts through character-level tokenization, this byte-level formulation avoids dataset-specific cues and provides a more rigorous basis for evaluating ciphertext learnability. Fig. 8 represents the different pre-processing methods between both encryption methods.

A. AES ENCRYPTION

AES encryption uses a 256-bit key and a 16-byte initialization vector (IV), ensuring a unique encryption outcome for each key and IV pair. The uniqueness of AES lies in its ability to obscure any recognizable patterns within the payload data by yielding an unrepeatable outcome for every unique combination of key and IV. This guarantees that even if the same packet data is encrypted multiple times, the outcomes will differ each time when a different IV is used. After encryption, the AES-encrypted packets are converted into a hexadecimal format. Each character in this hexadecimal representation is then mapped to its corresponding decimal integer, forming a sequence that is padded to uniform sequence lengths across the dataset. The M.L. model's average test accuracy with AES-encrypted data was 50.40% with an F1-Score of 50.40%. Similar results were observed when testing other previously mentioned SOA methods, none of which achieved an accuracy greater than 50% in classifying malicious or benign packets. These metrics

indicate that AES encryption substantially diminishes the model's ability to discern between malicious and benign payloads.

B. FERNET ENCRYPTION

Fernet employs AES in Cipher Block Chaining (CBC) mode with a 128-bit key. Fernet's encrypted outputs are base64-encoded, resulting in a more diverse character set. Similar to AES, the encrypted payloads under Fernet encryption are also converted into sequences of decimal integers. However, due to the base64 encoding, each character is first converted to its ASCII value before the sequence is formed. This process allows for a larger vocabulary size since base64 encoding includes a broader range of characters. The application of Fernet encryption resulted in test accuracy of 51.64% and an F1-Score of 51.57%. Once again, when tested with other SOA classifiers, the model fails to distinguish malicious from benign content with better than random guessing performance. This suggests Fernet effectively conceals patterns within the data by pairing strong AES encryption with Base64 encoding, therefore leaving deep-learning methods with no contextual cues for reliable detection.

XI. DISCUSSION AND LIMITATIONS

A. DISCUSSION

Our research stands out in several key ways. We began by removing duplicated packet byte values, enhancing the model's exposure to unique packets and thereby improving data quality for more effective generalization to the testing dataset. Retaining these duplicates can reduce the uniqueness of signatures, making the process for classification algorithms easier to identify which packets belong to a certain class. Another pivotal aspect of our approach is the strict focus on packet bytes as inputs. Previous classification methods typically rely on statistical-based feature methodologies, such as size of the packet, port numbers, flow duration, etc., derived from traffic data. While this approach is sufficient for traditional machine learning algorithms, sequence-based inputs, such as byte sequences, benefit significantly more from modeling long-term dependencies. Our analysis was also not limited to the initial 784 payload bytes as described in [21], but extended to the MTU of 1500 bytes. Given that the typical MTU for Ethernet packet-based networks is 1500 bytes, with around 40 bytes allocated for header information, this approach ensures our model considers the full length of available payload data. This is important as malicious content might be embedded further within a packet's payload. By removing the constraint of shortening the byte data, we address the broader complexity within in each payload.

In this study, it's important to acknowledge the distinct types of data used in each dataset, which reflect different network scenarios. The CIC-IOT23 dataset is comprised of network traffic from 105 devices within an IoT network

topology, designed to capture traffic patterns typical of IoT environments. However, the UNSW-NB15 dataset was constructed to provide a mix of benign activities and synthetic malicious behaviors, aiming to support a broad spectrum of network intrusion detection research. This dataset encompasses a variety of attack types and network behaviors, making it structurally and contextually different from the CIC-IOT23. The variation in network-connected devices across the datasets not only influences the nature of the network traffic but also highlights the unique operational scenarios each dataset is intended to simulate. Furthermore, the CIC-IOT23 dataset likely presents unique challenges and patterns that are not as prevalent in the UNSW-NB15 dataset.

An additional observation is that the simple ML classifier consistently outperforms more complex learning architectures such as CNNs and LSTMs when leveraging the SSL embeddings, despite its relatively lightweight structure. This can be explained by the strength of the SSL-generated embeddings, which already encode local and global dependencies within packet data. Since these embeddings are highly discriminative, the ML classifier can leverage them directly without requiring additional temporal feature extraction. In contrast, the deeper models may introduce unnecessary complexity or risk overfitting on the SSL embeddings. This finding demonstrates that once robust embeddings are learned, a simple downstream classifier can lead to improved accuracy and faster inference, which is especially beneficial for real-time deployment.

The practical implementation entails deploying the malware detection model onto edge devices within the network layer. The edge devices are responsible for connecting different segments of the network, enabling traffic between networks through a physical component. The malware detection model would be implemented directly on these devices, allowing for the immediate analysis of network traffic as it passes through. This placement means that the model can efficiently filter and examine data packets in real-time, identifying potential threats at the network's edge. Also, this model could be deployed across numerous edge devices to provide comprehensive coverage of a network.

B. LIMITATIONS AND THREATS TO VALIDITY

Our study demonstrates promising results, but several limitations and threats to validity should be acknowledged. Pre-training with transformers is computationally expensive and depends heavily on dataset scale [48], [49]. Larger and more diverse datasets generally yield richer embeddings but require substantial compute and training time, which may limit accessibility for organizations without high-performance infrastructure. However, the growing availability of high-performance GPUs is making large-scale pre-training on datasets of benign and malicious behaviors increasingly feasible. In addition, because our SSL embeddings are fixed after pre-training, downstream tasks are constrained by the representations learned from the

pre-training dataset. This can limit the model's ability to continuously generalize to novel malware families over time. While our proposed FSL method partially mitigates this, future work could investigate incremental or online SSL training to refine embeddings in real time. With parameter-efficient fine-tuning methods such as LoRA [50]), this becomes increasingly practical, since only a small set of additive parameters must be updated rather than fully fine-tuning the entire model.

Furthermore, our approach assumes access to plaintext transport-layer payloads for DPI. In environments with widespread encryption, payload observability is reduced, and our model performs at chance when packet bytes are fully encrypted using Fernet or AES. We note earlier in the manuscript that DPI often presumes access to unencrypted payloads [30], [31], a condition that can still hold in certain enterprise networks, industrial control systems (ICS), and CAN applications. Future work will combine payload-agnostic statistics with our SSL embeddings and evaluate on further encrypted captures.

We trained and evaluated on two public corpora (UNSW-NB15 and CIC-IoT23) with deliberate class selection and de-duplication to balance classes; nonetheless, focusing on a limited set of attack families per corpus and sampling subsets can introduce bias in byte distributions and background traffic characteristics. While performance and few-shot transfer indicate promise within the chosen attacks across these two datasets, external validity remains limited to these settings. To strengthen generalization claims, future work will conduct cross-generalization capabilities of the proposed model, incorporate additional captures spanning network traffic collection pipelines, and evaluate domain-adaptation and deployability strategies for out-of-distribution real-time malware detection.

XII. CONCLUSION AND FUTURE RESEARCH DIRECTIONS

This paper explored the application of a self-supervised transformer-based model with a downstream task and few-shot learning for malware detection and classification. The model was evaluated on the UNSW-NB15 and CIC-IOT23 datasets, focusing on the packets of UDP and TCP packets serving as inputs. Our method showed robust results on the classification of benign versus malicious packets when compared to SOA methods on both datasets. Furthermore, our results show that a simple ML classifier, when paired with SSL-generated embeddings, can outperform more complex CNN and LSTM models. This emphasizes that SSL pre-training results in discriminative embeddings, allowing lightweight models to achieve improved accuracy and efficiency compared to deeper architectures, which is advantageous for real-time intrusion detection. The few-shot results were also promising, demonstrating the model's ability to learn with limited amounts of labeled data. Using the packet bytes as input vectors, we were able to classify when different packets are either benign or malicious, as well as which types of attacks were present. While

the payload bytes of packets are inherently different from the natural language structure, this study shows that the transformer-based model developed for natural language can be leveraged to capture and learn the intricate sequential patterns of the packet bytes.

Despite these encouraging results, several challenges and opportunities remain. Although our evaluation focused on traditional enterprise network and IoT packets, it is crucial to extend out work to additional protocols across several domains. In particular, protocols associated with ICS and in-vehicle CAN present unique characteristics which may require additional pre-training. Furthermore, another promising direction is the design and implementation of new models based off our proposed transformer-based architecture that incorporates emerging paradigms such as continual learning, which would enable rapid adaptation to evolving traffic patterns while remaining suitable for deployment in resource-constrained environments. Lastly, building attribution models that leverage distinctive network packet signatures represents an important challenge, as such models would enhance interpretability by linking malicious behaviors to identifiable network traits. Overall, this work proposed a self-supervised transformer-based pre-training framework for malware detection and classification, demonstrating its versatility through multiple downstream tasks, including FSL.

ACKNOWLEDGMENT

This work is partially supported by the UWF Argo Cyber Emerging Scholars (ACES) program funded by the National Science Foundation (NSF) CyberCorps® Scholarship for Service (SFS) award under grant number 1946442. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the authors and do not necessarily reflect the views of the NSF.

REFERENCES

- [1] H. J. Kiratsata, D. P. Raval, P. K. Viras, P. Lalwani, H. R. Patel, and S. Panchal, "Behaviour analysis of open-source firewalls under security crisis," in *Proc. Int. Conf. Wireless Commun. Signal Process. Netw. (WiSPNET)*, 2022, pp. 105–109.
- [2] C. Rohith and G. Kaur, "A comprehensive study on malware detection and prevention techniques used by anti-virus," in *Proc. 2nd Int. Conf. Intell. Eng. Manage. (ICIEM)*, Apr. 2021, pp. 429–434.
- [3] O. N. Nyasore, P. Zavarisky, B. Swar, R. Naiyeju, and S. Dabra, "Deep packet inspection in industrial automation control system to mitigate attacks exploiting modbus/TCP vulnerabilities," in *Proc. IEEE IEEE 6th Int. Conf. Big Data Secur. Cloud (BigDataSecurity) Int. Conf. High Perform. Smart Comput., (HPSC) IEEE Int. Conf. Intell. Data Secur. (IDS)*, May 2020, pp. 241–245.
- [4] G. Aceto, D. Ciunzio, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2022, pp. 5998–6008.
- [8] *Malware Statistics*. Accessed: Jan. 20, 2024. [Online]. Available: <https://dataprot.net/statistics/malware-statistics/>
- [9] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A deep learning approach for network intrusion detection system," in *Proc. 9th EAI Int. Conf. Bio-Inspired Inf. Commun. Technol. (Formerly BIONETICS)*, 2016, pp. 21–26.
- [10] R. Doshi, N. Aphorpe, and N. Feamster, "Machine learning DDoS detection for consumer Internet of Things devices," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2018, pp. 29–35.
- [11] J. G. Almaraz-Rivera, J. A. Perez-Diaz, and J. A. Cantoral-Ceballos, "Transport and application layer DDoS attacks detection to IoT devices by using machine learning and deep learning models," *Sensors*, vol. 22, no. 9, p. 3367, Apr. 2022.
- [12] K. Stein, A. Mahyari, and E. El-Sheikh, "Vehicle controller area network inspection using recurrent neural networks," in *Proc. Int. Conf. Adv. Comput. Res.*, 2023, pp. 494–499.
- [13] M. F. Sayeed and I. Al-Anbagi, "Deep ensemble learning with pruning for DDoS attack detection in IoT networks," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 2, pp. 596–616, 2024.
- [14] Y. He, X. Kang, Q. Yan, and E. Li, "ResNeXt+: Attention mechanisms based on ResNeXt for malware detection and classification," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1142–1155, 2024.
- [15] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.
- [16] V. Ravi and M. Alazab, "Attention-based convolutional neural network deep learning approach for robust malware classification," *Comput. Intell.*, vol. 39, no. 1, pp. 145–168, Feb. 2023.
- [17] Y. Yu and N. Bian, "An intrusion detection method using few-shot learning," *IEEE Access*, vol. 8, pp. 49730–49740, 2020.
- [18] C. Lu, X. Wang, A. Yang, Y. Liu, and Z. Dong, "A few-shot-based model-agnostic meta-learning for intrusion detection in security of Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21309–21321, Dec. 2023.
- [19] X. Zhou, W. Liang, S. Shimizu, J. Ma, and Q. Jin, "Siamese neural network based few-shot learning for anomaly detection in industrial cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5790–5798, Aug. 2021.
- [20] K. Stein, A. A. Mahyari, G. Francia, and E. El-Sheikh, "Towards novel malicious packet recognition: A few-shot learning approach," in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, Oct. 2024, pp. 847–852.
- [21] M. D. Lucia, P. Maxwell, N. D. Bastian, A. Swami, B. Jalaian, and N. Leslie, "Machine learning raw network traffic detection," *Proc. SPIE*, vol. 11746, pp. 185–194, Apr. 2021.
- [22] L. Xu, X. Zhou, Y. Ren, and Y. Qin, "A traffic classification method based on packet transport layer payload by ensemble learning," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2019, pp. 1–6.
- [23] J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, "A survey on self-supervised learning: Algorithms, applications, and future trends," 2023, *arXiv:2301.05712*.
- [24] X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang, "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
- [25] Rahul, A. Gupta, A. Raj, and M. Arora, "IP traffic classification of 4G network using machine learning techniques," in *Proc. 5th Int. Conf. Comput. Methodologies Commun. (ICCMC)*, Apr. 2021, pp. 127–132.
- [26] B. Zhang, W. Xiao, X. Xiao, A. K. Sangaiah, W. Zhang, and J. Zhang, "Ransomware classification using patch-based CNN and self-attention network on embedded N-grams of opcodes," *Future Gener. Comput. Syst.*, vol. 110, pp. 708–720, Sep. 2020.
- [27] A. Andriekute. (2023). *TCP Vs UDP: What's the Main Difference?*. [Online]. Available: <https://nordvpn.com/blog/tcp-or-udp-which-is-better/>
- [28] S. Kumar and S. Rai, "Survey on transport layer protocols: TCP & UDP," *Int. J. Comput. Appl.*, vol. 46, no. 7, pp. 20–25, 2012.
- [29] Y. A. Farrukh, I. Khan, S. Wali, D. Bierbrauer, J. A. Pavlik, and N. D. Bastian, "Payload-byte: A tool for extracting and labeling packet capture files of modern network intrusion detection datasets," in *Proc. IEEE/ACM Int. Conf. Big Data Comput.*, Dec. 2022, pp. 58–67.
- [30] L. Deri and F. Fusco, "Using deep packet inspection in CyberTraffic analysis," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience (CSR)*, Jul. 2021, pp. 89–94.

- [31] A. Dijk, "Detection of advanced persistent threats using artificial intelligence for deep packet inspection," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2021, pp. 2092–2097.
- [32] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2022, pp. 3637–3645.
- [33] J. Snell, K. Swersky, and R. S. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 1–12.
- [34] I. M. Ziko, J. Dolz, É. Granger, and I. B. Ayed, "Laplacian regularized few-shot learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 11660–11670.
- [35] K. Stein, A. A. Mahyari, G. Francia, and E. El-Sheikh, "Transductive oneshot learning meet subspace decomposition," in *IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2025, pp. 576–581.
- [36] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.
- [37] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023.
- [38] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6.
- [39] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [40] (2023). *Main Classes-Optimizer and Schedules*. [Online]. Available: https://huggingface.co/docs/transformers/main_classes/optimizer_schedules
- [41] S. Laenen and L. Bertinetto, "On episodes, prototypical networks, and few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2022, pp. 24581–24592.
- [42] S. Ali, O. Abusabha, F. Ali, M. Imran, and T. Abuhmed, "Effective multitask deep learning for IoT malware detection and identification using behavioral traffic analysis," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1199–1209, Jun. 2023.
- [43] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL, USA: CRC Press, 2018.
- [44] F. Al-Obaidy, S. Momtahn, M. F. Hossain, and F. Mohammadi, "Encrypted traffic classification based ML for identifying different social media applications," in *Proc. IEEE Can. Conf. Electr. Comput. Eng. (CCECE)*, May 2019, pp. 1–5.
- [45] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [46] PyCryptodome. (2021). *AES-Advanced Encryption Standard*. [Online]. Available: <https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html>
- [47] (2021). *Fernet (Symmetric Encryption)*. [Online]. Available: <https://cryptography.io/en/latest/fernet/>
- [48] J. Beal, H. Wu, D. H. Park, A. Zhai, and D. Kislyuk, "Billion-scale pretraining with vision transformers for multi-task visual representations," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Jun. 2022, pp. 564–573.
- [49] J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, "A survey on self-supervised learning: Algorithms, applications, and future trends," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9052–9071, Dec. 2024.
- [50] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022, p. 3.



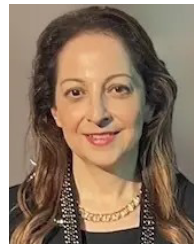
KYLE STEIN (Student Member, IEEE) received the B.S. degree in interdisciplinary natural sciences from the University of South Florida, Tampa, FL, USA, in 2018, and the M.S. degree in mathematical sciences from the University of West Florida, Pensacola, FL, USA, in 2021, where he is currently pursuing the Ph.D. degree in intelligent systems and robotics supported by the CyberCorps Scholarship for Service (SFS).

From 2024 to 2025, he interned at the Johns Hopkins University Applied Physics Laboratory. Previous to this experience, he was a Research Intern with the Science and Technology Directorate of the U.S. Department of Homeland Security, in 2023. His research focuses on low-shot learning, unseen threat detection, and deep learning applications in computer vision and cybersecurity. He is an Affiliated Student with Florida Institute for Human and Machine Cognition.



GUILLERMO FRANCIA III (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in computer science from New Mexico Tech. He joined the Center for Cybersecurity, University of West Florida, in 2018. Previously, he was the Director of the Center for Information Security and Assurance and held a distinguished professor position at Jacksonville State University. He received the Fulbright Scholarship Award: Teaching Award in Malta, in 2007, and the Research Award, U.K.,

in 2017. He was a recipient of numerous cybersecurity research and curriculum development grants.



EMAN EL-SHEIKH (Member, IEEE) received the M.Sc. and Ph.D. degrees in computer science from Michigan State University. She is currently a Professor of computer science at the University of West Florida and the Associate Vice President and a Professor at the Center for Cybersecurity, University of West Florida. She is the Chief Strategic Alliance Officer and an USA Ambassador of the Global Council for Responsible AI.



ANDREW ARASH MAHYARI (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in electrical engineering and the M.Sc. degree in statistics from Michigan State University, in 2016. From 2017 to 2018, he was at the ABB Robotics Research and Development Center, San Jose, CA, USA. He is currently a Research Scientist at Florida Institute for Human and Machine Cognition and a Research Professor at the Department of Intelligent Systems and Robotics, University of West Florida.

...